Lab 3

Fundamental Software Design Principles

Student: Albert Muratbaev

Part A - Modularity and information hiding

Q1. One job, one sentence

Item	One module or two	The modules and their jobs
(a)	Two	Timetable: stores the lessons of a group. ReminderSender: sends SMS reminders to the students.
(b)	One	Grades: computes the final mark of a student.
(c)	Two	Auth: checks a password. LoginScreen: draws the login screen.
(d)	One	Export: turns a finished report into a PDF file.

Q2. Cut a system into modules

• Session: keeps the film session and its seat map.

• Seats: shows which seats of the chosen session are free.

• Ticket: sells a ticket for a session and records the sale.

• TicketPrint: prints a sold ticket with its QR code.

• SalesReport: at night counts how many tickets each film sold.

Arrows: Seats uses Session; Ticket uses Session; TicketPrint uses Ticket; SalesReport uses Ticket.

Module with the most arrows pointing at it: Ticket (two arrows: from TicketPrint and from SalesReport). This is not a problem by itself: it is the central business object and other modules naturally record or read what it sells. It becomes a problem only if Ticket keeps growing (pricing, printing, reporting inside it); then every small change touches many modules.

Q3. Where is the border wrong?

Case	Broken rule	Where the border should be
(a)	A module must own one job (a piece of knowledge), not one kind of syntax.	Loops and checks belong inside the modules whose data they serve; no separate loops.py and checks.py.
(b)	Data must have one owner; nobody changes another module's data directly.	balance lives in an Account module that exposes deposit() and withdraw(); Booking, Refund and Admin call these methods.
(c)	One external dependency must have one boundary.	A Telegram module wraps the API; Booking, Report and Admin call it instead of the API.
(d)	Things that change together belong together.	Price and tax are one tariff rule set; they live in one Pricing module, tax.py is merged into it.

Q4. A parking lot with no secrets

Which coupling: content coupling - outside code reads and writes the internal fields of Parking.

After places becomes a dict {place_number: car_number}:

Outside line	Still correct?	Why
parking.places[12] = "01A123BC"	Looks correct	Writing a dict entry happens to match the new representation, but it still skips any booking check.
if None in parking.places:	Wrong, and dangerous	The dict holds place numbers as keys, never None, so the condition is always false: the screen says there is a free place even when the parking is full.
hours * parking.prices["car"]	Correct	The price table did not change.

Most dangerous: the free-place check. It does not crash; it silently gives a wrong answer, so users keep trusting a lie - a loud crash, by contrast, is noticed and fixed at once.

Public interface: book(place_number, car_number) -> bool; free_place() -> place number or None; leave(car_number) -> bool; price(vehicle_type, hours) -> sum.

Two secrets: (1) how the places are stored (list, dict or anything else); (2) the price table and how the cost is computed.

Q5. Just give me the list

No: get_places() would hand out the internal representation and create content coupling with every screen. Instead I would add free_places() -> int (or is_free(n)), which returns only the answer the screen needs.

Part B - Abstraction

Q6. Three words students confuse

One word for each sentence:

Sentence	Word
(a)	encapsulation
(b)	information hiding
(c)	abstraction
(d)	information hiding
(e)	abstraction

Q7. Procedural or data abstraction?

Item	P or D	What the caller does not know
(a) queue enqueue/dequeue	D	How the queue stores its elements (list, linked cells...).
(b) print_certificate	P	How the certificate is laid out and produced.
(c) backup_database	P	How the backup is made and where copies are stored.
(d) phonebook add/find	D	The internal phonebook structure.
(e) convert_to_pdf	P	How the PDF file is generated.
(f) cart add/total	D	How the cart stores and computes its items.

Q8. Put the levels in order

Order from the user (highest) to the hardware (lowest): C, D, F, A, E, B.

Level	One thing it knows	One thing it does not know
C - the user taps Send	That the user wants to send this text to this chat	That the message will travel as JSON, bytes or radio waves
D - chat.send(chat_id, text)	Which chat and text the user chose	How the network or the message record is built
F - Message.create(sender_id, text)	The structure of a message	Where the message physically goes
A - http.post(/messages, json)	The URL, the JSON body and status codes	Who the user is or what a chat is
E - socket.send(bytes)	Ports, packets, bytes	URLs, JSON, users, chats
B - the Wi-Fi chip	Radio signals and frames	Everything about the chat application

Q9. One interface, two implementations

Nothing in the program that uses the interface must change; only the line that constructs a Counter is rewritten. The two versions have identical method signatures.

With one million words and 1,000 count() calls, version 2 wins. Version 1 scans the whole list for every count(), about 1,000 x 1,000,000 comparisons in total; version 2 keeps a ready dict, so each count() is a single lookup and the 1,000 calls cost

almost nothing.

The interface promises that the answer is correct. It says nothing about speed. This is exactly what lets the implementer choose or replace the implementation: as long as every caller gets the correct answer, the team may trade speed, memory and simplicity inside the class without breaking anyone.

Q10. How much should the interface show?

(a) too much: everyone reads and writes the inside of the timer, including the admin writing timer.running = False.

(b) too little: the screen team must rebuild the remaining-time rule from the start time; that knowledge belongs to the timer.

(c) just enough: start(minutes), stop(), seconds_left() and is_finished() cover all the needs with no internals exposed.

In (b) the rule broken a few months later is DRY: the remaining time is now computed in two places (the timer and the screen code). People notice when the two copies disagree - for example after an exam rule change one screen shows the wrong time.

Test of slide 24 applied to (c): tonight I could replace the stored countdown by a recomputation from the start timestamp, and I could change how is_finished() is decided (for example from a recorded end moment), and no caller would have to hear about it, because both details are hidden behind the same four methods.

Part C - Cohesion

Q11. Name the cohesion

Piece	Level	How I recognized it
(a) make_thumbnail	Sequential	Each step consumes the result of the previous one: load -> resize -> grayscale -> save.
(b) end_of_day	Temporal	Unrelated jobs that happen at the same moment (closing, backup, report, lights).
(c) is_leap_year	Functional	One tiny job and nothing else.
(d) class Misc	Coincidental	Three unrelated helpers only share the file name.
(e) passport_summary	Communicational	All three calls work on the same passport; no pipeline between them.
(f) export(report, fmt)	Logical	Same kind of work with one branch chosen by a flag.
(g) register_for_exam	Procedural	Fixed steps of one workflow, but the steps share no data.

Q12. Three versions, three levels

A: sequential. B: communicational - every call works on the same card; the order of the warnings does not matter. C: procedural.

The one question: is the output of one line the input of the next line? If yes - sequential. If no, but all the lines work on the same piece of data - communicational. If the lines have a fixed order but do not pass data to each other - procedural.

Difference: temporal means the parts only belong together because they run at the same moment and share nothing; procedural means the parts are steps of one workflow that would run in this order even in the middle of the day. Version C is procedural: open- header- date- close is a sequence of the printing job, not a nightly coincidence.

Q13. The and-test on a real class

One sentence: Invoice adds lines, computes the total, converts itself into a PDF, e-mails the customer and saves itself into the database - the word and appears three times, so the class has at least four or five jobs.

Change	Method that must be edited
(a) the e-mail provider changes	email_to_customer
(b) the tax rate changes	total (and often add_line)
(c) the database moves to PostgreSQL	save_to_db
(d) the PDF must show the shop logo	to_pdf

Number of different reasons to change: four (at least).

Split (names and methods only):

• Invoice: add_line(product, qty); total()

• PdfFormatter: to_pdf(invoice)

• CustomerEmailer: send(invoice, address)

• InvoiceRepository: save(invoice)

Q14. Read the signals

Signal	Evidence in the story
The module is too long	1,400 lines and about a hundred more every week.
Many unrelated dependencies	It imports sqlite3, smtplib, reportlab and requests - four different worlds in one file.
Methods that do not share data	Half of the methods use self.connection, the other half use self.smtp and never touch the database.
Many people change it for different reasons	In one month Aziza fixed the PDF, Bekzod added an SMS provider, Dilnoza fixed a SQL query.
A meaningless name	DataProcessor says nothing about one job.

Strongest signal: the split between self.connection and self.smtp users. Check in thirty seconds: search the file for which methods mention connection and which mention smtp - if the two sets of method names do not overlap, the class is really two classes.

Q15. When is temporal cohesion acceptable?

It is acceptable when the only thing the parts have in common is the moment they run, and each job itself lives in its own cohesive module: end_of_day() is then just a scheduled trigger, like slide 6 - timing does not stop the jobs from being separated. It is not acceptable when the jobs themselves are mixed inside the same function.

Two concrete problems of the second version: (1) the business rule (summing the day's sales) is written inline in the driver, so a change of the rule touches a night job; (2) storage and infrastructure are hard-coded (sqlite3.connect, smtp.send, lights.off), so changing the database or the message system means editing this one function that also turns off the lights - exactly the mix that temporal cohesion is supposed to avoid.

Part D - Coupling

Q16. Name the coupling

Case	Coupling
(a) send_sms(user)	Stamp - the whole User with 15 fields travels, only phone is used.
(b) bookings.csv with ;	External - two modules depend on the same outside file format.
(c) area(width, height)	Data - exactly the values needed are passed.
(d) db._connection...	Content - Report reaches inside Database and uses its private part.
(e) CURRENT_USER global	Common - both functions read and write the same global data.
(f) format_price(amount, style)	Control - the style flag picks the branch inside the function.

From the worst to the best: d (content), e (common), b (external), f (control), a (stamp), c (data).

Q17. Move one step down the table

(a) Stamp -> data:

• def send_sms(phone): sms_gateway.send(phone, "Your code: 4821")

• call: send_sms(user.phone)

(e) Common -> data:

• def login(name): return Session(name)

• def can_delete(session): return session.name == "admin"

(f) Control -> data:

• def format_price_short(amount): return f"{amount} sum"

• def format_price_long(amount): return f"{amount:,} UZS"

(d) Content -> data:

• Database gets a public method: get_sales() -> rows

• def build(self, db): rows = db.get_sales()

Q18. The same boolean, two kinds of coupling

create_user(name, is_admin) is data coupling: True/False becomes a stored attribute of the new user; no branch inside changes meaning. save(user, to_cloud) is control coupling: the flag chooses which of two branches inside save() will run.

A reader of save(user, True) does not know what True means - upload where? overwrite what? The call says nothing about the target.

Rewrite without a flag:

• def save(user, storage): storage.write(user)

• storage is a CloudStorage or a DiskStorage object; both offer write(user) - no branch inside save.

Q19. Who decides the messenger?

Three decisions AttendanceReport makes that are not its job: (1) it chooses the messenger (Telegram); (2) it owns the secret token; (3) it names the recipient @group_ce22.

Wish 1 (send by e-mail): the constructor and the body must be rewritten to call the e-mail library; Wish 2 (tests send nothing): a test must succeed in stopping the sending - usually an if inside send(), which pushes test knowledge into production code.

Rewrite:

• class AttendanceReport: def __init__(self, sender): ... def send(self, text): self.sender.post(text)

• for real use: AttendanceReport(TelegramBot("TOKEN-123"))

• for a test: AttendanceReport(NullSender()) - a sender that never contacts any network.

Q20. Place it on the golden-rule square

Design	Cohesion	Coupling	Corner
(a) System, 3,000 lines, everyone imports it	low	high	the worst corner
(b) five small classes + global SETTINGS	high	high	high cohesion, high coupling
(c) three unrelated utils files	low	low	low cohesion, low coupling
(d) O receives Pricing, Storage, Receipt	high	low	the goal corner

Change that moves (b) to the goal corner: make one Settings module the owner of the dictionary and expose get(name)/set(name, value); the five classes then receive the values they need as ordinary parameters (or call the single owner), so no global shared state is left.

Part E - Separation of concerns, DRY, KISS, YAGNI

Q21. Which rule is broken?

Case	Rule broken	One-line repair
(a)	DRY	One constant MAX_STUDENTS and one full-limit message live in group.py; enroll.py and messages.py use them.
(b)	KISS	Write return age >= 18.
(c)	YAGNI	Build one role now; add the twelfth when a second real role appears.
(d)	Separation of concerns	Compute the price in a rule function, save through a repository, show the result in the screen - three separate lines/calls.
(e)	KISS	Write max(a, b, c).

Q22. Same code, different knowledge

I do not fully agree. The two functions serve different concerns with different owners and names (shipping vs parking), so fee(x) would replace two clear names with one flag-driven function - but the repeated knowledge inside them, the tariff 2000, is a real DRY item.

What to do with 2000: name it in each owning module - UZS_PER_KG and UZS_PER_HOUR - or hold one named tariff constant in one place if the university really prices both with the same table; the number itself must never be typed twice without a name.

Q23. YAGNI or simple good design?

Developer B breaks YAGNI; developer A is doing good design. A pays almost nothing today (one messages.py) and removes a real cost later: when a second language does come, the texts are already out of the code and moving them is cheap. B pays for an engine, a picker and a download system today, with seven languages to test and maintain, while the customer asked for one language - the cost is real now and the saving never arrives, because features nobody asked for usually never get used.

Q24. Sort the lines by concern

Line	Concern
1 if balance < price: refuse()	Business rules
2 INSERT INTO orders ...	Storage
3 show_message(...)	Presentation
4 logger.info(...)	Infrastructure
5 discount = 0.1 if has_meal_card ...	Business rules
6 sms.send(phone, ...)	Infrastructure
7 button.color = green	Presentation
8 SELECT balance FROM wallets ...	Storage

A class named OrderScreen may contain only the Presentation concern (showing results, collecting input); it must not write SQL, decide rules or send SMS itself.

Part F - Case study: the dormitory laundry

Q25. Count the jobs

Jobs of book(): (1) checks whether the machine is free; (2) writes the busy state into the global STATUS; (3) computes the price from the mode flag; (4) applies the resident discount; (5) appends the booking to bookings.txt; (6) sends the Telegram notification; (7) prints a confirmation - five or six jobs depending on whether pricing and discount are counted as one.

Level of cohesion: coincidental (dragged together only by the booking event), with a control flag for the mode; reason: checking, pricing, storage, notification and printing are unrelated jobs.

Three different numbers are written directly inside book(): 8000, 6000 and 0.10 - and 8000 is written twice, in the wash and the wash+dry branch.

Q26. Name every coupling

Item	Coupling
(a) the global STATUS	Common coupling
(b) the parameter mode	Control coupling
(c) bookings.txt with ; (also read by WeeklyReport)	External coupling - two modules depend on the same file format
(d) the whole student object (only 3 of its fields used)	Stamp coupling
(e) telegram.send(...) inside book()	External coupling to a concrete device/service

Q27. Who owns STATUS?

Rustam books machine 3 at 10:00; the entry stays forever, so Malika at 18:00 is refused with Machine is busy. Tomorrow it is the same: nothing ever clears STATUS and book() never looks at the hour, so machine 3 is dead until the program is restarted.

The data must be owned by a schedule module (rule 2 of slide 15: one owner of the data). Call it MachineSchedule.

Public interface of MachineSchedule:

• book(machine, hour, student) -> bool

• is_booked(machine, hour) -> bool

• release(machine, hour) -> bool

• bookings(machine) -> list

After the change nobody else may write to that data: MachineSchedule is the single writer; every other module goes through its methods (WeeklyReport may read, but only through a reader method or repository).

Q28. Redesign on paper

Classes (names and methods only):

• MachineSchedule: book(machine, hour, student) -> bool; is_booked(machine, hour); bookings(machine)

• PriceCalculator: price(mode, is_resident) -> sum

• BookingRepository: append(booking); read_all() -> lines

• TelegramNotifier: notify(telegram_id, text)

• BookingService (the connector): receives schedule, calculator, repository, notifier in its constructor; book(student, machine, hour, mode) -> receipt

Boxes and arrows: BookingService points to the four classes; WeeklyReport points to BookingRepository. That is five arrows; counting them stops me from adding a sixth dependency (for example calling Telegram directly from the schedule).

Where the numbers live now: 8000, 6000 and 0.10 all live inside PriceCalculator, and each is written exactly once.

After the redesign the confirmation Booked! Price: ... is printed by the caller of BookingService - the screen or presentation module; the service itself only returns the price in a receipt.

Q29. The two numbers

Change	Old design: what is edited	New design: which classes change
(a) discount 15%	The number 0.10 inside book()	PriceCalculator only
(b) SMS instead of Telegram	The telegram.send call inside book() and the imports	Replace the Notifier used by BookingService (one class, no service change)
(c) SQLite instead of bookings.txt	book() file code AND WeeklyReport	Add SqliteBookingRepository; the service is unchanged; WeeklyReport reads through the repository
(d) test with no messages and no files	Nothing clean is possible in the old design - the test must fake telegram, files and stdout	Pass NullNotifier and an in-memory repository into BookingService in the test

Why only one class changes in the old design is not good news: that one class is the laundry class that holds all the jobs, so every change is made inside the same crowded body, the file is a shared point of conflict for every future wish, and nothing about the change can be tested in isolation.

Part G - Coupling in depth

Q30. The settings everybody reads

Coupling between settings.py and each of the three: common coupling - all four share one mutable global dictionary.

Pairs that depend on each other without importing: cart.py and invoice.py (their outputs must agree: the total rounded in the cart and the digits printed on the invoice lines), and both of them with admin.py, because admin silently redefines the values they already computed with.

Two wrong things the customer can see after switch_to_usd() at 10:01: (1) totals of carts that were already filled at 10:00 are printed in USD but rounded with 0 decimals - a soum amount displayed as whole dollars; (2) the cart and the invoice of the same order can disagree, because each module rounds at the moment it reads the shared dictionary.

The one thing inside switch_to_usd() that allows an inconsistent read: the change is not atomic - it writes two keys in two separate statements, so an unlucky reader can observe currency = USD together with decimals = 0, a pair that does not belong together.

If decimals must become a rounding step, cart.py and invoice.py must be edited (round(sum, n) and the .{n}f format both assume whole digits); if one module owned the whole rule, zero outside files would change.

The owner (Prices) exposes at most four methods:

• round(amount) -> number

• format(amount) -> string

• currency() -> string

• set_rule(currency, step) - called only by the admin

After the change cart.py and invoice.py know nothing about currency: they handle plain amounts and call the owner for rounding and formatting. Only the owner (through set_rule, called by the admin function) may change the currency.

Q31. Data that only travels

All three calls (checkout -> confirm -> schedule) are stamp coupling: each passes the whole Order while using none of it (checkout, confirm) or only two fields (schedule). The functions that only carry data they do not use are checkout and confirm.

When order.address becomes order.addresses: only schedule() must change, because it is the only function that touches the address; checkout and confirm are unchanged. The function that should have been the only one to know about addresses is schedule (and behind it delivery_date receives city and weight as plain values already).

Rewrite so each function receives only what it uses:

• def checkout(payment) -> confirm(payment) - 1 parameter

• def confirm(payment) -> schedule(payment) - 1 parameter

• def schedule(city, weight_kg) -> delivery_date(city, weight_kg) - 2 parameters

• def delivery_date(city, weight_kg) - 2 parameters

(In real code checkout and confirm, which add nothing, would collapse and disappear.)

Choosing between six values and the Order object: six separate values keep data coupling but the call contracts grow - every time Order gains a field somebody must update the parameter lists. Passing the Order is stamp coupling - schedule and confirm stay aware of the whole type and break whenever its shape changes. Cost of data choice: long parameter lists and their maintenance; cost of stamp choice: the whole call chain is welded to the Order class.

Q32. Fewer arrows, more surprises

Version 1 knows 3 modules (fines, notifier, stats); version 2 knows 1 abstraction (its listeners).

Version 2 still couples to its listeners by data coupling - they read named fields of the same Loan object. If Loan loses the field they read, all three listeners break at the same call (the first loop iteration raises AttributeError inside return_book).

Order: version 1 shows it literally in the body (close -> check -> tell -> record). Version 2 hides it inside the subscription order. A bug possible only in version 2: an out-of-order or duplicated subscription, for example a listener subscribed twice or after the notifier, so the next reader is told the book is free before the fine has been recorded, or receives two notifications.

Two listeners that never change: keep version 1 - three explicit lines are cheaper to write, read and debug than a subscription mechanism. Twelve listeners written by five teams: version 2 - adding a listener then requires no change to Loans, so the teams do not edit, build and conflict on one shared module.

Q33. The loop of imports

Import arrows: orders -> inventory (reserve), inventory -> pricing (stock_value), pricing -> orders (Order). To test one file alone it must be importable without the others: orders needs inventory and pricing; inventory needs pricing; pricing needs orders - and through that, all three. Because of the loop, in practice none of them can be tested alone.

To ship pricing.py as a separate package, the Order class must travel with it (a copy or a shared base/protocol), because pricing imports it.

First way - move one function: move stock_value() from pricing.py into inventory.py (inventory still can do its own logging with it):

• orders -> inventory -> (nothing); pricing -> orders. Loop broken; test order: inventory alone, orders with inventory, pricing with orders.

Second way - change what discount() receives:

• def discount(level): return 0.1 if level == "gold" else 0.0

• arrows: orders -> inventory -> pricing -> (nothing); the caller passes order.customer.level.

Order from knows nothing to knows the most (second way): pricing, inventory, orders. This gives the build a fixed bottom-up sequence and gives each module unit tests that run against nothing but the layer below it - in a loop no such order exists, so the build and the tests are interleaved and cannot be isolated.

Part H - Separation of concerns, DRY, KISS, YAGNI in depth

Q34. One rule, five places

Piece of knowledge	Files holding it
A number is valid: +998 and 9 more digits, 13 characters	registration.py (the regex), sms.py (len == 13), schema.sql (code + length), test_data.py (13-character examples)
A number has a leading + that must be stripped for the external gateway	sms.py
Public display rule: first 4 shown, rest masked	receipt.py
Two concrete valid formats as examples	test_data.py

No line is copied - the knowledge is repeated.

Kazakhstan (+7, 12 characters): four places must be edited (registration.py, sms.py, schema.sql, test_data.py). The one that will be forgotten is test_data.py, because it is the least visible; the error shows up later as tests that still pass on old numbers while valid new numbers are rejected in production - or vice versa, valid production numbers rejected by an old test.

Owner module PhoneNumbers:

• is_valid(phone) -> bool

• digits_only(phone) -> string

• mask(phone) -> string

• COUNTRY_CODE (constant)

The place that cannot use it: schema.sql - the database check is plain SQL and cannot import a module, so the CHECK stays a deliberate copy.

Is the remaining database check a DRY violation? No - a deliberate copy at a trust boundary is correct: the database protects against every future writer (other services, manual inserts), not just this module. Add one test that pushes a valid and an invalid number through both the module and the schema, so the two copies cannot silently drift apart.

Q35. DRY gone too far

The flags create control coupling; the cohesion of calculate() is coincidental - nothing holds its parts together except one entry point; a caller can ask for 2^4 = 16 behaviours, the three callers use 3.

The author followed DRY (one function for all variants) and broke KISS (a four-flag switchboard instead of clear names) and separation of concerns (pricing, tax, rounding and notification in one body).

Redesign - the smallest shared piece and the three callers on top:

• def sum_products(items): return sum(i.price * i.qty for i in items) # the only shared piece

• def quote(items): return sum_products(items)

• def invoice(items): return round_up(apply_tax(sum_products(items)))

• def refund(items): return -apply_tax(sum_products(items))

• notification is a separate call made by whoever needs it; flags left: zero

• (a) two functions checking the 13-character phone: extract one shared validator - it is the same knowledge about one format; (b) two four-line loops summing prices and weights: keep both - they are unrelated copies of a pattern that will

diverge; extracting saves nothing and costs a shared dependency.

Q36. When two principles disagree

Situation	Disagreeing principles	My choice	The fact that would make me change sides
(a) 40-line VAT in two services	DRY vs decoupling / release independence	Keep two copies: sharing welds two releases together	If the law required the invoices of both services to use one identical VAT value at every moment, one shared owner would be right
(b) 15-line script split into five classes	KISS vs separation of concerns	Keep the script	If it becomes a monthly process several people edit and must not break, the split earns its cost
(c) slow get(key) on a million entries	Information hiding vs performance	Keep the secret; fix performance inside	If no internal change can be fast enough, the Cache has to publish a new bulk operation - an interface change, not a public list

(c) change that keeps the secret and removes the slowness: add one public bulk method, for example read_entries(batch_size) or an internal index, that reads a million entries through a controlled method while the storage layout stays private.

Q37. The arrows point the wrong way

Import arrows: screens -> rules -> storage (the natural direction), and storage -> screens - the wrong arrow: Storage knows the presentation and raises an Exception carrying Uzbek error text.

Concerns inside the wrong modules: storage.py contains presentation (the Uzbek message); rules.py contains presentation (HTML with <p class='red'>); screens.py adds none of its own.

Telegram bot in Russian: the new bot module must copy the call structure and the knowledge of the cases (found/not found, debt) because the business result is only available as Uzbek-shaped HTML; it either parses that HTML or repeats the rule work with its own Russian answer.

Redesign (no text, no HTML):

• storage.find_student(student_id) -> row/dict or raises StudentNotFound

• rules.check_in(student_id, room) -> Allowed(room) or Denied(reason) with machine reasons like 'debt'

• screens.check_in_page(form) -> maps Allowed/Denied to Uzbek HTML; the bot maps the same verdict to Russian

The texts now live in the presentation modules themselves (screens.py for Uzbek, the bot for Russian); the business rules return only reason codes.

Arrows after the change: screens -> rules -> storage = 2 arrows. rules.py may know about storage; screens.py may know about rules (and, on that allowed direction, storage as the layer below); storage.py may know about none of the others.

Part I - Case study: the library fines

Q38. Audit the code

Evidence - copying the exact problem lines for five of the seven areas:

• Cohesion: the whole return_book method - it checks, computes, discounts, caps, stores, e-mails and prints in one body.

• Coupling: FINES = [] # global - common coupling (plus the CSV contract with MonthlyReport).

• Separation of concerns: the rule (days_late * 500, gold / 2, cap by book.price), the file write and email.send(...) live in one method.

• DRY: the CSV format written here (reader.name + ',' + ... + str(fine)) and separately re-invented by MonthlyReport's line.split(',')).

• YAGNI: total_fines() - nothing in the story or the code ever calls it.

Two areas with no evidence in this code: information hiding and KISS. Information hiding: no outside module touches the private work of Library - nothing is exposed beyond the two methods and the shared files, so there is no line to cite. KISS: every individual rule line (fine = days_late * 500, gold / 2, the cap) is as simple as the rule itself - the mess of this code is cohesion, coupling and duplicated knowledge, not over-complex logic. I do not invent a problem for them because an audit that stretches for missing evidence would itself be dishonest - the honest answer is that these two principles are currently satisfied.

A book titled Algorithms, Part 1: the stored line contains a comma inside the title, so MonthlyReport's line.split(',') produces more than three parts and the report crashes with a ValueError (too many values to unpack). Kind of coupling: external - two modules depend on the same outside file format. Which module is at fault and why neither is better: the Library writes without quoting and MonthlyReport splits without escaping, so each module is locally reasonable; the real fault is the missing owner of the storage format - a rule (no unquoted commas, or a quoted CSV) that belongs to a design decision written nowhere, so blaming one module is less accurate than saying the format has no owner.

Restart difference: FINES is an in-process list, so after a restart total_fines() sums from zero while month_total('fines.csv') still counts every stored fine - two truths about one data set. Root cause: DRY (the same data is stored in two different places, one of which is not durable).

Three changes in an order that keeps the program working after every step:

• 1. Write the fine record with proper quoting (csv.writer or a separator that cannot appear in titles), and read it the same way in MonthlyReport - this alone fixes the crash for titles like Algorithms, Part 1, which is the most visible failure today.

• 2. Make one store the single source of truth: the file (via a small Ledger/Repository) is written and read by everyone; FINES is removed or rebuilt from the store, so total_fines() and the report can never disagree after a restart.

• 3. Keep money as whole sum integers: replace fine = fine / 2 with an integer operation (for example (days_late * 500) // 2 capped by the book price) so no receipt ever shows fractional sum.

Q39. Two designs, five changes

Change	Design X (4 classes)	Design Y (11 classes)
(a) gold discount 30%	edit FineCalculator	edit GoldDiscountRule
(b) new silver card 80%	edit FineCalculator	add SilverRule and register it in RuleChain
(c) fines in SQLite	edit FineLedger	add SqliteFineRepository, change the wiring in Library
(d) first three days free	edit FineCalculator	edit DaysLateRule (or add FreeDaysRule to RuleChain)
(e) also send an SMS	edit Notifier	add SmsChannel next to EmailChannel, change the wiring

Arrows. X: Library -> FineCalculator, Library -> FineLedger, Library -> Notifier, MonthlyReport -> FineLedger = 4 arrows. Y: Library -> FineService; FineService -> RuleChain, FineRepository, NotificationChannel; the three rules -> FineRule; CsvFineRepository -> FineRepository; EmailChannel -> NotificationChannel; MonthlyReport -> CsvFineRepository - about 11 arrows counting every implementation-to-interface edge.

Owner of the format of a stored fine: in X it is FineLedger (one visible class); in Y it is CsvFineRepository - hidden behind the FineRepository interface.

I hand in Design X. The principle of Part E that Y breaks is YAGNI (with KISS right behind it): a rule-engine of interfaces and five rule classes is built for fine rules that are today one short formula. The customer's sentence that shows it: the gold discount becomes 30 percent - in X this is one number in one class, while Y's engine was never asked for. The one fact about the library that would move me toward Y: if new, quickly-changing fine rules from many independent people kept arriving every semester (staff cards, category exemptions, seasonal rules), the plugin structure would start paying for itself.